



Plugins

Stream Deck Data Provider

Stream Deck Data Provider

Plugin for integrating HTML widgets with the iCUE Stream Deck provider: connects to Stream Deck devices, manages virtual device layout, and sends key events.

Overview

- **Module name:** `widgetbuilder.streamdeck`
- **Plugin name:** `StreamDeck`
- **Version:** `1.0`

Manifest entry:

```
"required_plugins": [  
  "widgetbuilder.streamdeck:StreamDeck:1.0"  
]
```

Runtime objects:

- Initialization flag: `pluginStreamdeck_initialized`
- Initialization callback map: `pluginStreamdeckEvents = { "onInitialized": onStreamdeckInitialized }`
- Plugin access: `window.plugins.Streamdeck`

All methods are synchronous from JavaScript perspective but operate on an asynchronous RPC connection to the Stream Deck provider. Device creation, icon updates, and error conditions are delivered via signals.

Methods

```
connectStreamDeck(widgetId, deviceId, columns, rows)
```

Connects the widget to a Stream Deck device and creates a virtual device with a given grid size.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Unique widget identifier provided by iCUE
<code>deviceId</code>	<code>string</code>	Target Stream Deck device identifier
<code>columns</code>	<code>int</code>	Number of columns in the virtual key matrix
<code>rows</code>	<code>int</code>	Number of rows in the virtual key matrix

On success, `virtualDeviceCreated(widgetId, deviceId)` is emitted once the virtual device is ready.

`reconnectStreamDeck(widgetId)`

Requests reconnection for a previously connected widget. The method is asynchronous and does not create a new virtual device by itself; it only restarts the underlying connection logic for the widget.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier used during connect

Use this when the Stream Deck process was restarted or the connection was lost and re-established.

Signal behavior:

- If the existing virtual device for `widgetId` is still present on the Stream Deck side, `virtualDeviceCreated(widgetId, deviceId)` is not emitted again. The original `deviceId` remains valid. When the device is opened after reconnection, the plugin will re-emit `buttonIconUpdated(widgetId, buttonIndex, iconDataUrl)` for all known buttons.

- If the stored virtual device no longer exists (for example, the Stream Deck app lost its state), a **new virtual device is created** and `virtualDeviceCreated(widgetId, deviceId)` is emitted again with the new `deviceId`.
- In case of connection problems or authentication issues, no special "reconnect failed" signal is emitted; instead, the regular error signals are used:
 - `streamdeckUnreachable(widgetId)` when the connection transitions to the Disconnected state;
 - `authenticationRequired(widgetId)` when user authentication is needed;
 - `authenticationRejected(widgetId)` if authentication fails.

`updateVirtualDeviceSize(widgetId, columns, rows)`

Updates the size of the virtual device grid without reconnecting.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier
<code>columns</code>	<code>int</code>	New number of columns
<code>rows</code>	<code>int</code>	New number of rows

The provider keeps existing connection and only resizes the virtual device.

`sendKeyPress(widgetId, buttonIndex, pressed)`

Sends a key press or release event for a virtual button.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier
<code>buttonIndex</code>	<code>int</code>	Zero-based button index in the virtual key grid

Parameter	Type	Description
<code>pressed</code>	<code>bool</code>	<code>true</code> for press, <code>false</code> for release

The grid is traversed in row-major order:

- Rows are numbered from top to bottom starting at `0`.
- Columns are numbered from left to right starting at `0`.
- The index is computed as:

```
buttonIndex = row * columns + column
```

For example, for a `5 × 3` grid (`columns = 5`, `rows = 3`):

- `buttonIndex = 0` → row `0`, column `0` (top-left)
- `buttonIndex = 4` → row `0`, column `4` (top-right)
- `buttonIndex = 5` → row `1`, column `0` (first button of the second row)
- `buttonIndex = 14` → row `2`, column `4` (bottom-right)

Use this from an interactive widget when the user taps or releases a virtual key.

Signals

`virtualDeviceCreated(widgetId, deviceId)`

Emitted when a virtual Stream Deck device has been created and is ready.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier
<code>deviceId</code>	<code>string</code>	Connected Stream Deck device id

`buttonIconUpdated(widgetId, buttonIndex, iconDataUrl)`

Emitted when an icon for a specific virtual button changes.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier
<code>buttonIndex</code>	<code>int</code>	Button index whose icon was updated
<code>iconDataUrl</code>	<code>string</code>	Icon image as a <code>data:</code> URL (PNG, etc)

`streamdeckUnreachable(widgetId)`

Emitted when the Stream Deck provider cannot be reached or the connection is lost.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier

`authenticationRequired(widgetId)`

Emitted when authentication is required before the widget can interact with Stream Deck.

Parameter	Type	Description
<code>widgetId</code>	<code>string</code>	Widget identifier

`authenticationRejected(widgetId)`

Emitted when authentication fails or is rejected.

Parameter	Type	Description
widgetId	string	Widget identifier

Example

manifest.json

```
"required_plugins": [  
  "widgetbuilder.streamdeck:StreamDeck:1.0"  
]
```

index.html

```
// Initialize plugin events (optional, for async signals)
pluginStreamdeckEvents = {
    onInitialized: function onStreamdeckInitialized() {
        // Plugin is ready; you can safely call methods here
        connectStreamDeck();
    }
};

function connectStreamDeck() {
    if (window.plugins && window.plugins.Streamdeck && pluginStreamdeck_initialized)
    {
        const widgetId = iCUE.widgetId;
        const deviceId = iCUE.streamDeckDeviceId; // Provided by iCUE environment
        const columns = 5;
        const rows = 3;

        window.plugins.Streamdeck.connectStreamDeck(widgetId, deviceId, columns,
rows);
    }
}

function onVirtualButtonPressed(buttonIndex, pressed) {
    if (window.plugins && window.plugins.Streamdeck && pluginStreamdeck_initialized)
    {
        window.plugins.Streamdeck.sendKeyPress(iCUE.widgetId, buttonIndex, pressed);
    }
}
```

StreamDeck QML Wrapper

The StreamDeck QML wrapper provides a thin, declarative layer over the HTML plugin API so that QML components can control Stream Deck devices without writing JavaScript glue code.

The wrapper is implemented in the `widgetbuilder.streamdeck` module as `StreamDeck.qml` and internally forwards all calls to the same backend that powers the HTML plugin.

Initialization

To use the QML wrapper, import the module and keep a single `StreamDeck` object in the QML scope that needs to interact with Stream Deck. The wrapper exposes the same methods and signals as the HTML plugin:

- `connectStreamDeck(widgetId, deviceId, columns, rows)`
- `reconnectStreamDeck(widgetId)`
- `disconnectStreamDeck(widgetId)`
- `updateVirtualDeviceSize(widgetId, columns, rows)`
- `sendKeyPress(widgetId, buttonIndex, pressed)`

Signals:

- `virtualDeviceCreated(widgetId, deviceId)`
- `buttonIconUpdated(widgetId, buttonIndex, iconDataUrl)`
- `streamdeckUnreachable(widgetId)`
- `authenticationRequired(widgetId)`
- `authenticationRejected(widgetId)`

QML Example


```
import QtQuick
import widgetbuilder.streamdeck

Item {
    id: root

    property string widgetId: "{widget-id-from-environment}"
    property string deviceId: "{streamdeck-device-id-from-environment}"

    StreamDeck {
        id: streamDeck

        onVirtualDeviceCreated: function(widgetId, deviceId) {
            // Handle initial virtual device creation
        }

        onButtonIconUpdated: function(widgetId, buttonIndex, iconDataUrl) {
            // Update button visuals in QML if needed
        }
    }

    Component.onCompleted: {
        streamDeck.connectStreamDeck(widgetId, deviceId, 5, 3);
    }
}
```